

MODULE 3

AI/ML-Based Predictive Maintenance and Vehicle Health Monitoring System

Submitted by

Name: Rohit Roy

Roll No.: 41

Enrolment No.:12023002002115

Wipro Automotive Course

Electronics and Communication Engineering

Institute Of Engineering and Management, Newtown

Under the guidance of

Rabi Behera

Date of submission: 29.09.2023

Overview

This report presents a predictive-maintenance solution that uses vehicle operating data to estimate the likelihood of failure before a breakdown occurs. The workflow combines conventional machine-learning classifiers with sequence-based deep-learning models, then translates the model output into a practical health score, condition category, maintenance recommendation, and explanation of influential operating factors.

The study uses a synthetic dataset of 10,000 observations and nine vehicle parameters. The implementation is intended as a proof of concept; results should be validated with real sensor and service-history data before operational deployment.

1. Introduction

Unexpected vehicle or component failures can lead to unplanned downtime, higher repair costs, safety concerns, and reduced fleet availability. Conventional maintenance schedules are often based on fixed intervals and may not reflect the actual condition of a vehicle. Predictive maintenance addresses this limitation by examining operating data and identifying patterns that may precede a failure.

The central problem addressed in this project is the development of a model-assisted monitoring workflow that can process vehicle parameters, estimate failure probability, and communicate the result in a form that is easy for a technician or operator to understand.

2. Project Objectives

- Prepare a structured dataset containing key vehicle operating variables.
- Clean and transform the data into a form suitable for predictive modelling.
- Develop and compare classical machine-learning and deep-learning approaches.
- Estimate the probability of an impending vehicle or component failure.
- Convert the prediction into a health score and operating condition category.
- Provide maintenance guidance and identify the parameters that most influenced the prediction.

3. Proposed System Architecture

The solution follows a clear analytical pipeline:

Vehicle sensors → Data ingestion → Preprocessing → Predictive model → Failure probability → Health score → Condition status → Maintenance recommendation

The system accepts nine operating measurements. After preprocessing, the selected model estimates the probability of failure. That probability is then converted into a health score and a simple decision label so the output can support maintenance planning rather than remain a raw model prediction.

Stage	Purpose
Data acquisition	Collect operating measurements from the vehicle or a historical dataset.
Preprocessing	Select features, check data quality, scale inputs where required, and form sequences for temporal models.
Prediction	Estimate the probability of a failure event.

Decision layer	Calculate health score, classify condition, and generate an alert.
Interpretation	Highlight influential variables and provide a maintenance-oriented explanation.

4. Dataset and Input Variables

The project dataset contains 10,000 records and 10 columns. It was synthetically generated for development and testing. The dataset contains no missing values. The target variable is Failure, where 0 represents no recorded failure and 1 represents a failure event. The class distribution is 9,609 non-failure records and 391 failure records, so the dataset is imbalanced and recall should be considered carefully during evaluation.

Variable	Description	Use
Engine Temperature	Thermal condition of the engine	Input
RPM	Engine rotational speed	Input
Oil Pressure	Lubrication-system pressure	Input
Vibration	Mechanical vibration level	Input
Battery Voltage	Electrical-system voltage	Input
Coolant Temperature	Cooling-system temperature	Input
Fuel Consumption	Fuel usage rate	Input
Vehicle Speed	Vehicle operating speed	Input
Operating Hours	Accumulated operating duration	Input
Failure	Failure-event label	Target

5. Data Preparation

- The nine vehicle measurements were used as explanatory variables, while Failure was used as the target.
- The data was separated into training and testing subsets for model development and assessment.
- Standardization was applied to models that benefit from comparable feature scales.
- Sequences of length 10 were prepared for LSTM, GRU, and Transformer experiments so that nearby observations could be analysed as a short operating history.
- Because failure cases are less frequent than non-failure cases, accuracy alone is not sufficient for judging model usefulness.

6. Predictive Models

Classical machine learning

Logistic Regression was used as a transparent baseline. Decision Tree provided rule-based classification. Random Forest combined multiple trees to improve robustness and offered feature-importance information. XGBoost used boosted decision trees to model nonlinear relationships and was selected for the final monitoring function.

Deep learning

LSTM and GRU networks were applied to sequential vehicle data. A Transformer model using multi-head self-attention was also implemented to explore whether attention-based sequence modelling could capture relationships across the input window.

7. Evaluation Framework

The models were assessed using Accuracy, Precision, Recall, and F1 Score. Their detailed numerical values and comparison plots are available in the accompanying Google Colab notebook. In a predictive-maintenance setting, Recall is especially important because a missed failure can be more costly than an unnecessary inspection.

Model	Role in the study	Reported result
Logistic Regression	Baseline classifier	Refer to notebook output
Decision Tree	Rule-based classifier	Refer to notebook output
Random Forest	Ensemble classifier and importance analysis	Refer to notebook output
XGBoost	Final probability-estimation model	Refer to notebook output
LSTM	Sequence-learning model	Refer to notebook output
GRU	Efficient recurrent sequence model	Refer to notebook output
Transformer	Attention-based sequence model	Refer to notebook output

8. Vehicle Health Decision Layer

The deployed monitoring function receives the nine vehicle parameters and passes them to the trained XGBoost model. The model returns a failure probability. This value is displayed as a percentage and converted into a health score using:

Health Score = 100 – Failure Probability (%)

Health score	Condition	Recommended action
80% or above	HEALTHY	Continue normal operation and routine checks.
60% to below 80%	WARNING	Schedule an inspection and review abnormal parameters soon.
Below 60%	CRITICAL	Arrange immediate inspection before continued operation where appropriate.

These thresholds are practical interpretation rules for the prototype, not universally valid safety limits. In a production system, they should be calibrated against actual failure costs, maintenance procedures, and safety requirements.

9. Results and Interpretation

The completed prototype demonstrates an end-to-end predictive-maintenance workflow. It transforms vehicle measurements into model inputs, compares several predictive approaches, and produces an operational output consisting of failure probability, health score, condition status, maintenance guidance, and contributing factors.

XGBoost feature importance is used to identify influential variables. The system maps these variables to understandable explanations, such as elevated vibration, high engine or coolant temperature, abnormal oil pressure, low battery voltage, high fuel consumption, high RPM, high vehicle speed, or extended operating hours. These explanations should be treated as indicators for inspection rather than as proof that one component has failed.

10. Limitations and Future Development

The current study is a proof of concept based on synthetic data. Synthetic patterns may not represent the noise, sensor drift, missing readings, class imbalance, and complex failure modes found in real vehicles. In addition, the sequence models require carefully timestamped data, while the current report does not include the detailed numerical evaluation outputs from the notebook.

- Connect the system to real vehicle sensors and timestamped telemetry.
- Train and validate the models using maintenance records and verified failure events.
- Add sensor-quality checks, drift monitoring, and model-retraining procedures.
- Deploy the monitoring function through an IoT gateway or edge-computing device.
- Calibrate alert thresholds using safety, downtime, and maintenance-cost requirements.
- Add a dashboard for trends, alerts, service history, and technician feedback.

11. Technology Stack

- Python and Google Colab for development and experimentation.
- NumPy and Pandas for numerical processing and data preparation.
- Scikit-learn for classical machine-learning models and evaluation.
- XGBoost for boosted-tree classification.
- TensorFlow/Keras for LSTM, GRU, and Transformer models.
- Matplotlib and Seaborn for visual analysis and reporting.

12. Conclusion

This project establishes a practical foundation for AI-assisted vehicle maintenance. By combining sensor-oriented features, predictive models, and an interpretable decision layer, the system can move from a raw failure prediction to a clear maintenance-oriented recommendation. The next step is rigorous validation with real operational data, followed by threshold calibration and controlled deployment.

13. References

- [1] Scikit-learn Developers, “Scikit-learn: Machine learning in Python—Classification and model evaluation,” Scikit-learn Documentation. [Online]. Available: <https://scikit-learn.org/>
- [2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.
- [3] TensorFlow Developers, “TensorFlow/Keras documentation: Recurrent and attention-based neural networks,” TensorFlow Documentation. [Online]. Available: <https://www.tensorflow.org/>
- [4] Scikit-learn Developers, “sklearn.metrics: Classification metrics and model evaluation,” Scikit-learn Documentation. [Online]. Available: <https://scikit-learn.org/stable/api/sklearn.metrics.html>
- [5] XGBoost Developers, “XGBoost documentation,” XGBoost Documentation. [Online]. Available: <https://xgboost.readthedocs.io/>